



„Stefan cel Mare” University of Suceava
FACULTY OF ELECTRICAL ENGINEERING AND COMPUTER
SCIENCE
DUAL COMPUTER SYSTEMS PROGRAM

T H E I N V E N T O R Y

Inventory Management Microservice.

Project developed by:
Strugariu Filip-Daniel
FIESC Group: 3111Da Year 1

Suceava 2026

1 Table of contents	
2 Application description	3
3 API	3
4 Technologies / Tools.....	4
5 App structure	5
5.1 Database schema.....	5
5.2 Backend app structure (Module)	6
5.3 Frontend app structure	6
5.4 Git repo structure	6
6 App configuration & run.....	7
6.1 Local run (Development):.....	7
6.2 Production run	8
7 Conclusions / Lessons learned	8

2 Application description

The Inventory is a **Full-Stack** digital solution designed to optimize logistics workflows in medium-sized warehouses. The application allows real-time stock monitoring, supplier relationship management, and automated freight transfers between different physical locations.

Inspired by the standards of the **ASSIST Learning platform**, the project focuses on an intuitive user interface (**Dashboard**) and a high-performance backend capable of handling complex **CRUD** operations and rigorous data validations.

3 API

Automated documentation: the backend uses the **OpenAPI (Swagger)** specification for generating the testing interface.

Main routes:

/products: Product catalog management.

/warehouses: Storage capacity administration.

/inventory: Stock transfer and adjustment logic.

/suppliers: Commercial partners database.

The Inventory API 1.0.0 OAS 3.1

[/openapi.json](#)

Microservice for Inventory Management

Warehouse Management ^

GET	/api/warehouses	Get Warehouses	▼
POST	/api/warehouses	Create Warehouse	▼
GET	/api/warehouses/{id}	Get Warehouse	▼
PUT	/api/warehouses/{id}	Update Warehouse	▼
PATCH	/api/warehouses/{id}	Patch Warehouse	▼
DELETE	/api/warehouses/{id}	Delete Warehouse	▼

Supplier Management ^

GET	/api/suppliers	Get Suppliers	▼
POST	/api/suppliers	Create Supplier	▼
GET	/api/suppliers/{id}	Get Supplier	▼
PUT	/api/suppliers/{id}	Update Supplier	▼
PATCH	/api/suppliers/{id}	Patch Supplier	▼
DELETE	/api/suppliers/{id}	Delete Supplier	▼

Product Management			^
GET	/api/warehouses/{warehouseId}/products/all	Get Absolutely All Products	▼
POST	/api/warehouses/{warehouseId}/products	Create Product	▼
GET	/api/warehouses/{warehouseId}/products	Get All Products	▼
GET	/api/warehouses/{warehouseId}/products/{productId}	Get Product By Id	▼
PATCH	/api/warehouses/{warehouseId}/products/{productId}	Update Product Details	▼
PUT	/api/warehouses/{warehouseId}/products/{productId}	Update Or Create Product	▼
DELETE	/api/warehouses/{warehouseId}/products/{productId}	Delete Product	▼
Stock Management			^
GET	/api/warehouses/{warehouseId}/inventory	Get Inventory	▼
GET	/api/warehouses/{warehouseId}/inventory/{productId}	Get Product Inventory	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/increase	Increase Stock	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/decrease	Decrease Stock	▼
POST	/api/warehouses/{warehouseId}/inventory/{productId}/transfer	Transfer Stock	▼

4 Technologies / Tools

Python & FastAPI - Development of the backend microservice and business logic.

React.js & Vite - Building the frontend user interface and managing application state.

PostgreSQL (Supabase) - Persistent data storage in a cloud-based relational database.

SQLAlchemy - ORM used for mapping **Python** objects to database tables.

Tailwind CSS - Fast and responsive styling of frontend components.

Render - Live hosting for the backend **API**.

Hostinger - Production deployment of the frontend.

Visual Studio Code - Main Integrated Development Environment (**IDE**).

Swagger (OpenAPI) - Automated generation of interactive documentation for real-time **API** endpoint testing.

Pydantic - Defining data structures (schemas) and automated input validation to ensure database integrity.



5 App structure

5.1 Database schema

The system is built on a relational architecture:

- **Products:** Stores details about SKU, price, and name.
- **Warehouses:** Monitors location and maximum capacity.
- **Suppliers:** Manages the database of partners supplying the products, including contact details.



5.2 Backend app structure (Module)

- **main.py**: Entry point and **CORS** configuration.
database.py: Managing the connection to the **Supabase** instance.
- **models.py**: Defining the structure of **PostgreSQL** tables.
- **schemas.py**: Input/output data validation using **Pydantic**.

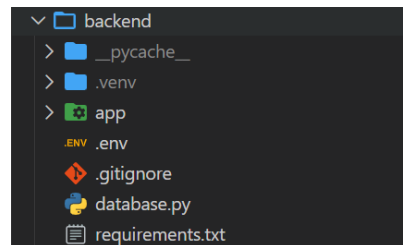
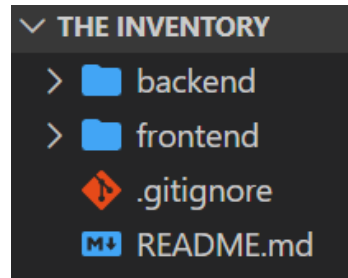
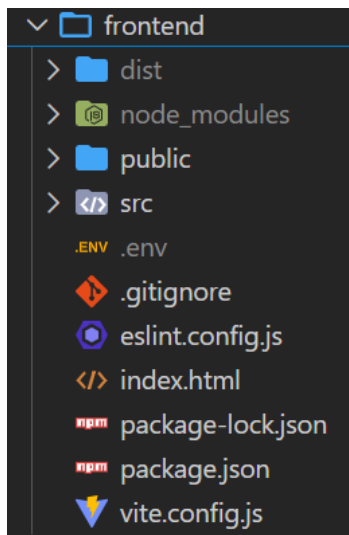
5.3 Frontend app structure

The interface is built in **React.js**, organized into reusable components:

- **UI components**: Dashboard, Inventory tables, Editing forms.
- **State management**: Managing data received from the **Render API** via **React** hooks (**useEffect**, **useState**).
- **Design**: Implemented with **Tailwind CSS** for a modern and responsive look.

5.4 Git repo structure

- **/backend**: Contains **Python** code, the **requirements.txt** file, and **API** logic.
- **/frontend**: Contains the **React** project, **Tailwind** configuration, and component folders.
- **.gitignore**: File extensions ignored by **Git**.
- **README.md**: The main documentation of the **repository**.



6 App configuration & run

6.1 Local run (Development):

```
backend > requirements.txt
1 annotated-doc==0.0.4
2 annotated-types==0.7.0
3 anyio==4.13.0
4 certifi==2026.2.25
5 charset-normalizer==3.4.6
6 click==8.3.1
7 colorama==0.4.6
8 fastapi==0.135.2
9 greenlet==3.3.2
10 h11==0.16.0
11 idna==3.11
12 psycopg2-binary==2.9.11
13 pydantic==2.12.5
14 pydantic_core==2.41.5
15 python-dotenv==1.2.2
16 requests==2.33.1
17 SQLAlchemy==2.0.48
18 starlette==1.0.0
19 typing-inspection==0.4.2
20 typing_extensions==4.15.0
21 urllib3==2.6.3
22 uvicorn==0.42.0
```

- **Backend:** Creating a virtual environment, installing dependencies (**pip install -r requirements.txt**), and starting the server with:

.\.venv\Scripts\activate

uvicorn app.main:app --reload.

```
(.venv) PS C:\Users\struf\Desktop\SITE\The Inventory\backend> .\.venv\Scripts\activate
(.venv) PS C:\Users\struf\Desktop\SITE\The Inventory\backend> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\struf\\Desktop\\SITE\\The Inventory\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [20228] using StatReload
INFO: Started server process [22388]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

- **Frontend:** Installing **Node** modules (**npm install**) and starting the development server (**npm run dev**)

```
PS C:\Users\struf\Desktop\SITE\The Inventory\frontend> npm run dev

> the-inventory-web@0.0.0 dev
> vite

Re-optimizing dependencies because vite config has changed

VITE v5.4.21 ready in 2408 ms

→ Local:   http://localhost:5173/The-inventory/
→ Network: use --host to expose
→ press h + enter to show help
```

6.2 Production run

- **Backend:** Automatically connected to **GitHub** via **Render** for **continuous deployment**..
- **Frontend:** Compiled (**npm run build**) and uploaded to **Hostinger**.



7 Conclusions / Lessons learned

By developing this project, I managed to deepen my understanding of data management mechanisms, learn how to create a scalable **API**-based infrastructure, and develop **Full-Stack** applications. Key knowledge acquired:

- **Data modeling:** I implemented a relational structure in **PostgreSQL** that ensures data consistency across products, warehouses, and suppliers, logically managing stocks without overloading the database with redundant tables.
- **Microservices development:** I used **FastAPI** to build an asynchronous **API**, focusing on data validation via **Pydantic** and securing stock transfer routes.
- **Cloud resource management:** I coordinated a distributed environment, learning to interconnect a secure cloud database (**Supabase**) with an **API** hosting service (**Render**).